Команда Сбера

Протоколы взаимодействия AI-агентов

Тращенков Сергей

Разработка и внедрение AI-агентов · МФТИ

Материалы

Telegram-канал

Какой протокол для какой задачи

MCP

Агент вызывает функцию сервиса: поиск поездов или продуктов.

A2A

Один агент поручает задачу другому и получает результат.

AG-UI

Приложение получает ход работы агента и показывает поиск, вызовы и части ответа.

A2UI

Приложение строит экран по описанию агента: карточку поезда с временем и ценой.

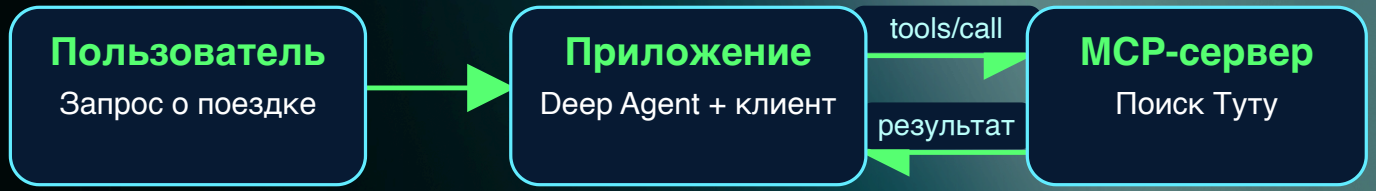
ACP: редактор кода отправляет агенту запросы и получает ответы и уведомления о выполнении.

Материалы

Telegram-канал

MCP: приложение, клиент и сервер

Приложение (host) создаёт MCP-клиента и передаёт модели описания инструментов.
Клиент отправляет вызов серверу, сервер выполняет операцию.



Материалы

Telegram-канал

MCP: инструменты, ресурсы и промпты

Tools: инструменты

Функции, которые клиент вызывает с аргументами.

`search_rail`: поиск поездов

Resources: ресурсы

Данные с адресом URI, например описание курса.

`course://overview`

Prompts: промпты

Шаблоны сообщений, например запрос справки по теме.

`prepare_lecture_brief`

Сервер предоставляет три вида возможностей. Приложение выбирает ресурс и промпт, а модель выбирает вызов инструмента.

Материалы Telegram-канал

MCP: как вызывается инструмент

Модель

Выбирает инструмент и задаёт аргументы

→

Приложение

Отправляет MCP-запрос

→

Сервер

Выполняет операцию

ToolMessage — сообщение LangChain с результатом инструмента. Оно входит в историю, которую модель получает на следующем шаге.

Материалы Telegram-канал

MCP: чтение ресурса по URI

Resource

Описание курса по URI

→

Приложение

Читает содержимое

→

Запрос к модели

Содержит текст ресурса

URI — адрес ресурса в MCP. Клиент читает course://overview, а приложение добавляет полученное описание курса в запрос к модели.

MCP: получение шаблона промпта

Команда Сбера

Аргументы

```
topic_name="mcp"
```

```
audience="студенты"
```

Результат сервера

«Подготовь справку по MCP для студентов; укажи назначение и пример применения»

Сервер подставляет тему и аудиторию в шаблон. Для получения ответа приложение отдельно вызывает модель с этими сообщениями.

MCP: ресурс и промпт в запросе к модели

Приложение собирает контекст: описание курса из resource, задачу из prompt и инструменты для чтения файлов.



Материалы

Telegram-канал

MCP: версии спецификации и Python SDK

Спецификация

`2025-11-25`
↓
`2026-07-28`

Python SDK

`v1` → `v2`

Версия Python-библиотеки

Спецификация задаёт формат обмена. Версия SDK определяет Python API для этого обмена.

Материалы Telegram-канал

MCP: начало обмена до и после обновления

Что передаётся	2025-11-25	2026-07-28
Версия и возможности	В отдельном initialize	В _meta каждого запроса
Идентификаторы состояния	HTTP-сессия Mcp-Session-Id	Идентификаторы объектов в аргументах
Сведения о сервере	В ответе на initialize	Через server/discover

В блокноте отправляем tools/call без initialize, затем проверяем подключение клиента предыдущей редакции.

MCP: сервер запрашивает недостающие данные

Сервер

input_required:
нужен ввод
пользователя

→

Приложение

Показывает вопрос
и получает ответ

→

Повтор запроса

inputResponses:
передаёт ответ
серверу

Так устроен многошаговый обмен MRTR: приложение получает недостающий ввод и повторяет вызов инструмента.

MCP Python SDK: переход с v1 на v2

Сервер

`mcp.server.fastmcp.FastMCP`

↓

`mcp.server.MCPServer`

Транспорт задаётся в run()

Клиент

```
from mcp import Client
```

async with сохраняет соединение до выхода из блока

Здесь меняется API официального Python SDK. Отдельная библиотека FastMCP используется адаптером LangChain.

MCP в LangChain: новый адаптер

Прежнее подключение	Используемое подключение
langchain-mcp-adapters	langchain[mcp]
MultiServerMCPClient	MCPAdapter
get_tools()	await adapter.list_tools()

MCPAdapter подключаем через async with. Ресурсы и промпты читаем клиентом MCP SDK. Модуль langchain.mcp имеет статус beta.

Материалы Telegram-канал

MCP Туту: список доступных инструментов

Подключение

URL MCP-сервера Туту

→

Список инструментов

list_tools()

→

Проверка

Имена, описания, схемы

Вызываем list_tools() и смотрим имена, описания и схемы аргументов до запуска агента.

Материалы Telegram-канал

MCP Туту: параметры инструмента в JSON Schema

Описание аргументов

Схема задаёт имена и типы аргументов. Список required указывает обязательные поля.

Допустимое значение null

```
anyOf: [integer, null]
```

API модели должен принимать такую схему

Допустимое null и необязательное поле — разные свойства схемы. GigaChat API v2 принимает используемые здесь схемы MCP.

Материалы Telegram-канал

MCP Туту: подключение к Deep Agent

```
tools = await adapter.list_tools()
agent = create_deep_agent(model=model, tools=tools)
result = await agent.ainvoke({"messages": messages})
```

> messages — список сообщений пользователя. ainvoke запускает агента: модель выбирает MCP-инструмент и получает результат его вызова.

MCP Туту: вызов и результат в истории

Запрос модели

AIMessage:
search_rail и аргументы

→

Результат вызова

ToolMessage:
ответ сервиса Туту

→

Ответ пользователю

AIMessage:
поезд, время и цена

Сопоставляем ответ агента с ToolMessage: дату, направление, отсутствие пересадок, время и цену.

MCP: сервер для чтения учебных материалов

Resources

Описание курса
Сведения о темах

Prompt

Справка по теме
Заданная аудитория

Tools

Список файлов
Чтение начала файла

Сервер читает файлы из data/course и предоставляет tools, resources и prompts для подготовки справки.

MCP по stdio: запуск сервера из клиента

MCP-клиент

Запускает Python

→

stdin / stdout

Сообщения JSON-RPC

→

Сервер

Читает учебные файлы

stdin и stdout — потоки ввода и вывода процесса. Клиент обменивается с сервером JSON-RPC и завершает его при выходе из async with.

Материалы Telegram-канал

MCP: агент готовит справку по теме

Контекст

Описание курса
и запрос справки

→

Инструменты

Агент находит файл
и читает его начало

→

Результат

Справка по теме
и история вызовов

В блокноте выводим сообщения перед вызовом модели, вызовы инструментов и готовую справку.

MCP по HTTP: запрос и ответ

Запускаем сервер учебных материалов по Streamable HTTP. В блокноте смотрим отправленный запрос и ответ.

Запрос

Метод tools/call
Аргументы операции
Версия и возможности в _meta

Ответ

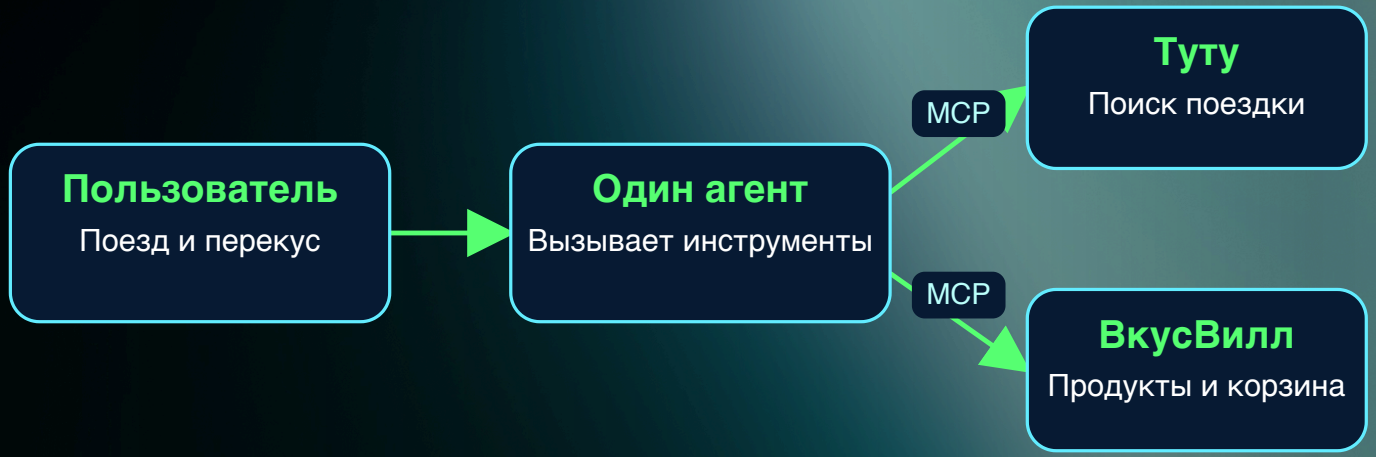
Результат операции
resultType: тип результата
Сведения о сервере

SSE — поток сообщений внутри HTTP-ответа. Streamable HTTP поддерживает такой ответ; запросы при этом остаются сообщениями MCP.

Материалы

Telegram-канал

MCP: один агент с Туту и ВкусВиллом



Один Deep Agent получает инструменты обоих MCP-серверов и выбирает вызовы для поиска поезда и продуктов.

MCP ВкусВилла: от поиска к корзине

Поиск

Названия и id продуктов

→

Выбор

3–4 позиции перекуса

→

Корзина

products: id + количество

Результат — ссылка на корзину с выбранными продуктами. Оформление и оплату оставляем пользователю.

Материалы

Telegram-канал

MCP и OAuth: доступ к личным данным

Приложение

Открывает страницу входа и разрешений

→

Пользователь

Подтверждает доступ

→

Сервис

Выдаёт токен доступа

OAuth позволяет пользователю разрешить приложению доступ и получить токен. В блокноте используются публичные операции без входа в личный кабинет.

Материалы Telegram-канал

Практика MCP: добавить инструмент на сервер

Добавить

Инструмент подсчёта учебных файлов по расширению

Проверить

Найти инструмент в list_tools
Вызвать клиентом
Попросить агента посчитать файлы

Доступ к файлам остаётся ограничен папкой data/course.

Команда Сбера

A2A, AG-UI, A2UI и ACP

Задачи между агентами, события для приложения и работа с редактором · 45 минут

Материалы

Telegram-канал

A2A: координатор и агент поездок



Координатор отправляет задачу по A2A. Агент поездок вызывает MCP Туту и возвращает результат; координатор подбирает продукты через MCP ВкусВилла.

A2A: адрес и возможности агента в Agent Card

Описание

Имя и назначение
Версия реализации

Подключение

URL сервера
Формат обмена
Версия протокола

Возможности

skills: задачи агента
Форматы входа и ответа
Поддержка потоковой выдачи

Agent Card — описание агента для клиента. По карточке клиент узнаёт, какие задачи можно отправить и по какому адресу.

A2A: сообщение, задача и результат

Запрос

SendMessage:
текст задачи

→

Выполнение

Task:
идентификатор и статус

→

Результат

Artifacts:
сведения о поездке

completed означает, что задача завершена. Artifact — результат задачи; в примере он содержит текст найденной поездки.

A2A: два сервера с одним агентом

Общая логика

Deep Agents + GigaChat
Один системный промпт
Одни инструменты Туту по MCP

Две реализации

Сервер на a2a-sdk

LangGraph Agent Server

Оба сервера запускают агента из agents/travel.py. Отправляем им одинаковый запрос одним клиентом.

A2A, способ А: сервер на a2a-sdk

Agent Card

Адрес сервера
и задачи агента

→

AgentExecutor

Вызывает Deep Agent
для поиска поездки

→

TaskUpdater

Обновляет статус
и добавляет результат

AgentExecutor получает A2A-запрос, вызывает агента и через TaskUpdater возвращает статус и результат.

A2A, способ B: LangGraph Agent Server

servers/agent_server.py

```
from agents.travel import travel_graph

graph = travel_graph
```

langgraph.json

```
{"graphs": {"travel": "./servers/agent_server.py:graph"}}
```

> langgraph.json указывает серверу функцию создания агента. Команда langgraph dev публикует его по A2A; MCP-соединение открыто во время поиска.

Материалы Telegram-канал

A2A: сравнение двух серверов

Проверяем	Оба варианта
Создание агента	Одна и та же функция
Источник данных	MCP Туту
Вход	Одинаковый запрос о поездке
Выход	Завершённая задача и текст в artifacts

Для обоих ответов проверяем дату, направление и соответствие данным Туту. Цена и формулировки могут меняться между запросами.

A2A: как клиент читает ответ

Запрос

Адрес из Agent Card
Метод SendMessage
Сообщение пользователя

Ответ

Идентификатор задачи
Статус выполнения
Текст в artifacts

Найдите эти поля в JSON-ответах обоих серверов. Клиент lab.ask_a2a использует их для чтения результата.

A2A и MCP: поездка и продукты в одном запросе

Агент поездок

Через A2A возвращает найденный поезд

→

Координатор

Через MCP подбирает продукты в дорогу

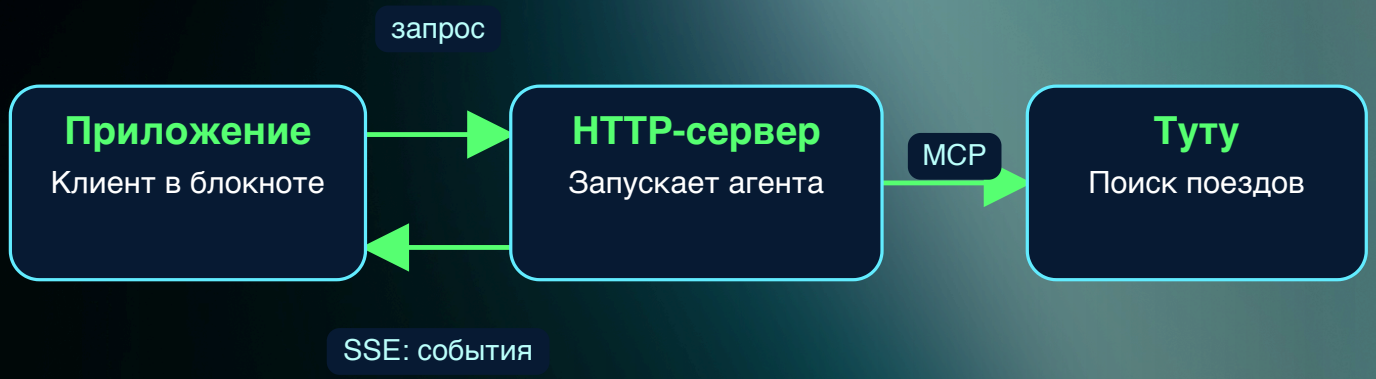
→

Общий ответ

Поезд и ссылка на корзину продуктов

Повторяем запрос с адресом второго A2A-сервера. Координатор по-прежнему получает поездку и ссылку на корзину.

AG-UI: события для приложения

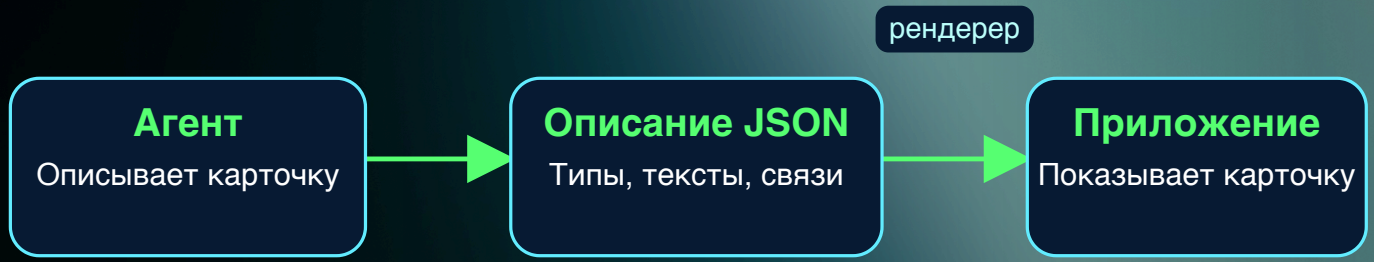


AG-UI передаёт приложению ход выполнения: агент начал работу, вызвал инструмент, дописал текст, завершил ответ. Блокнот получает эти события потоком SSE по HTTP.

AG-UI: как клиент отображает события

Событие	Действие клиента
RUN_STARTED	Показать, что агент начал работу
TOOL_CALL_START / TOOL_CALL_RESULT	Название инструмента и его результат
TEXT_MESSAGE_CONTENT	Дописать полученную часть ответа
RUN_FINISHED / RUN_ERROR	Завершение или сообщение об ошибке

A2UI: описание интерфейса



A2UI передаёт описание экрана: типы элементов, тексты и связи между ними. Рендерер — код приложения, который строит экран по этому описанию.

A2UI: от JSON к карточке поезда

Структура

createSurface: создать экран
updateComponents: задать элементы
Card → Text: текст внутри карточки

Данные

Номер поезда
Отправление и прибытие
Цена

В примере A2UI v0.9 описывает карточку с данными поезда. Блокнот показывает исходный JSON и полученное статическое изображение интерфейса.

ACP: редактор и агентный процесс



Agent Client Protocol связывает редактор кода с процессом агента. В примере роль редактора выполняет блокнот: отправляет запросы и получает ответы через stdin/stdout.

ACP: создание сессии и отправка запроса

initialize

Версия и возможности

→

session/new

Создать диалог и задать рабочую папку

→

session/prompt

Отправить задачу в эту сессию

session/update передаёт части ответа; stopReason=end_turn означает завершение. Сообщения JSON-RPC идут в stdout, диагностические сообщения — в stderr.

Покупка через агента: участники и согласие

Пример: агент подбирает продукты. Перед оплатой нужно согласовать состав заказа, сумму и разрешение пользователя.

Пользователь

Выбирает товар
и разрешает покупку

→

Агент и продавец

Уточняют покупку

→

Платёжный сервис

Проверяет
разрешение

Протоколы описывают обмен данными заказа и подтверждение права на оплату. В блокноте обсуждаем этот обмен без выполнения платежей.

Протоколы покупки и оплаты

Протокол	Назначение
Agentic Commerce Protocol	Состав, стоимость и оформление заказа между агентом и продавцом
UCP	Поиск товара, корзина, оформление и работа с заказом
AP2	Подтверждение полномочий агента и согласия пользователя на оплату
x402	HTTP-ответ 402 с условиями оплаты; повторный запрос с подтверждением

Agent Client Protocol и Agentic Commerce Protocol имеют одинаковую аббревиатуру ACP.

Материалы

Telegram-канал

Контрольные вопросы

Инструменты и контекст

Кто читает ресурс MCP?
Что возвращает промпт MCP?
Где выполняется инструмент?

Агенты и приложение

Кто ищет поездку по A2A?
Кто строит экран из A2UI?
Как ACP возвращает ответ?

Домашнее задание: RAG и MCP

Собрать

Свой корпус → локальные эмбеддинги
Индекс и готовый MCP-сервер поиска
Агент возвращает top-k и источники

Проверить и сдать

15–20 запросов и ручная оценка
document_id, chunk_id, source
Код, конфигурация и инструкция запуска

Полные условия и критерии: PDF задания в репозитории. По желанию: передать вопрос и top-k фрагменты в LLM; получить ответ со ссылками на источники.

Материалы

Telegram-канал

Команда Сбера

Skills: дополнительный материал

Пример навыка: создать QR-код, прочитать его и проверить содержимое

Skills: содержимое папки навыка

SKILL.md

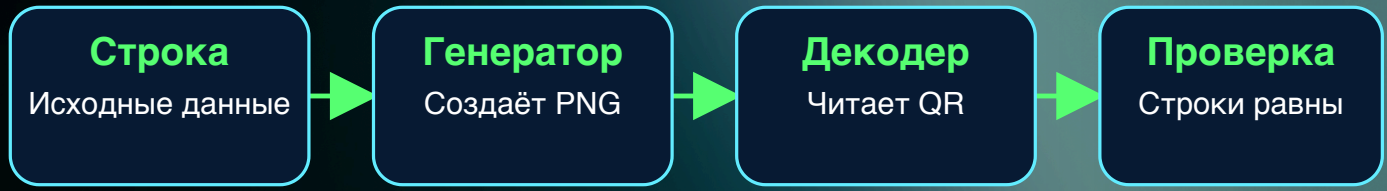
name: qr-code — как имя папки
description: когда применять
Инструкция: какие шаги выполнить

Файлы

scripts/qr_generate.py — создать PNG
scripts/qr_read.py — прочитать QR

По имени и описанию агент выбирает навык, читает SKILL.md и выполняет указанные шаги.

Skills: создание и чтение QR-кода



Агент создаёт QR-код и читает его обратно. В отдельной ячейке проверяем, что декодированная строка совпадает с исходной.

Skills: встроенный SkillsMiddleware

```
agent = create_deep_agent(
    model=model, backend=backend,
    skills=[skills_dir.as_posix()],
)
```

SkillsMiddleware

Имена, описания и пути к навыкам

→

read_file

Агент читает выбранный SKILL.md

→

execute

Агент запускает скрипты навыка

SkillsMiddleware показывает модели описания навыков. В блокноте проверьте, какой SKILL.md агент прочитал и какие команды выполнил.

Материалы Telegram-канал

Skills: установка QR-библиотек на разных ОС

ОС	Нативная библиотека
Windows x64	DLL в pyzbar; при ошибке — Visual C++ Runtime 2013 x64
macOS	brew install zbar
Debian/Ubuntu	apt-get install libzbar0

pyzbar — Python-обёртка над декодером ZBar. После установки выполните uv run python tools/check_env.py --qr: он создаёт и читает QR-код.

Спецификации и документация

MCP и инструменты

- Спецификация MCP
- MCP Python SDK v2
- MCP в LangChain

Агенты и интерфейсы

- Agent2Agent
- AG-UI
- A2UI
- Agent Client Protocol

Документация протоколов покупки и оплаты

- Agentic Commerce Protocol
- Universal Commerce Protocol
- Agent Payments Protocol
- x402

> Ссылки на документацию также приведены в блокноте.

Материалы

Telegram-канал